

What to do, in order

Every step of the project, from today to the defence. Phases A and B are finished and marked as such — what remains starts at Phase C.

PHASES	DONE	RIGHT NOW	PASSING BY
Seven, A to G	A and B, in full	Phase C, step 1	End of Phase D

1 · Orientation

How to use this document

Read one phase at a time. Not the whole thing. The phases after the one you are in are written down so they exist somewhere other than your head — that is their only job right now.

Every step has **what to do** and **how you know it is finished**. Steps already completed carry a **DONE** badge and a box giving the result that was actually obtained, so you can check the claim rather than trust it.

YOU ARE HERE

Phase C, step 1 — train the first agent

Phases A and B are complete. The simulator matches the car to **2.8 % load residual over 22 operating points**, drawn from 168 minutes of real driving across eight recordings.

Nothing past Phase D needs to be in your head. Phase D is the floor. Everything after it raises the ceiling.

Every number in this document is printed by a script. Run `python verify_docs.py` and it recomputes twenty-two of them from the shipped data and fails if any has drifted. If a figure here disagrees with that script, the script is right.

What is already done

Two of the seven phases are finished. This section exists so nobody repeats work that is already in the repository, and so the supervisor can see progress without reading the code.

2 · Status

Where the project actually stands

DRIVING DATA

168.1 min

8 recordings, 6 with usable samples

OPERATING POINTS

22

31–82 kPa, 1549–5053 rpm

LOAD RESIDUAL

2.8 %

target was under 15 %

PLANT VALIDATION

8 of 11

three misses, each explained

A	Get set up — repo, environments, first drive, the claim explained	DONE
B	Match the simulator to the car — validated on eight drives	DONE
C	Get an agent to learn — environment ready, nothing trained yet	NEXT
D	Baselines and the ablation — this is the floor	not started
E	Second plant — <code>battery.py</code> does not exist yet	not started
F	The H/τ sweep — preliminary result from hand-written policies	partial
G	Write it and defend it	not started

What Phase B produced, in one paragraph. A zero-dimensional cycle model of the B58B30O1 inline-six, validated two ways: against published figures (8 of 11 quantities inside band) and against the car itself (2.8 % load residual over 22 steady operating points). The baseline ECU’s spark map, its enrichment strategy and its knock retard are all calibrated against measured data rather than guessed. The three-node thermal network is calibrated against 80 minutes of logged coolant and oil.

Six corrections were made along the way, and they are the interesting part. The simulation ran the wrong engine for three weeks; the manifold-pressure channel is not manifold pressure; the baseline ECU was flattering the result; the enrichment map was fitted to the wrong variable; the air-mass sensor saturates; and a “60-second window” was neither 60 seconds nor a window. All are documented in `CLAUDE.md` with the evidence. An examiner will trust a project that reports against itself.

What happens if the ambitious half fails

THE MINIMUM PASSING PROJECT

Phases A to D, and nothing else

- 1. A simulator validated against a real car. **Done.** 2.8 % load residual over 22 points.
- 2. A learning agent that beats two baselines on a scenario that loads the engine.
- 3. An ablation showing the advantage comes from preview and nothing else.
- 4. A written account of all three, with the limitations stated.

That is a complete, defensible undergraduate project. Phases E and F make it a *contribution* rather than an *application*, and they are worth doing, but they are not what you pass on.

The ablation already works in miniature. `check_premise.py` races a hand-written predictive policy against a reactive one and then blinds the predictive one. Blinded, it lands on the reactive score *to the decimal* — 548.6 against 548.6. That identity has held across two different engines, two scenarios and two protection thresholds. The size of the gap has moved four times; the fact that it is attributable to preview has never moved.

The whole project on one page

A	Repo, environments, first drive, the claim explained to five people	done
B	Cycle model, thermal network, ECU calibration, all validated against the car	done
C	Reward sanity checks, then train one agent per person overnight	weeks 7–10
D	Three baselines, twenty fixed episodes, five seeds, the ablation table	weeks 11–14
E	<code>battery.py</code> with the same <code>predict()</code> contract	weeks 15–18
F	Sweep H/τ on both plants, produce one figure	weeks 19–24
G	Write, rehearse, defend	weeks 25–32

Get set up

Finished. Kept here because the gate condition is still the right one, and because a later team member joining the project starts at A1.

Phase A

Five steps — all complete

A1

Put the code somewhere five people can reach it DONE

A private repository, all five invited, the starter code committed.

FINISHED WHEN

All five have cloned it and each has pushed one commit.

A2

Everyone gets the same Python DONE

Python 3.11, a virtual environment, `pip install -r requirements.txt`, then everyone runs the premise check.

FINISHED WHEN

All five run `python check_premise.py` and see identical numbers.

RESULT

Current output, which is what five machines should now agree on: baseline damage **829.2**, reactive **548.6**, predictive **437.6**, preview-disabled **548.6**. **Any earlier set of numbers is void** — 527/357/199 and 52.7/29.9/27.8 were both measured before the engine geometry was corrected.

A3

Drive 1 DONE

The drive card, exactly as written: ten speed-and-gear conditions held as steadily as the road allows.

FINISHED WHEN

The CSV is exported and analysed.

RESULT

Eight recordings now exist, **168.1 minutes** in total. Two of them (`fb988991` , `f51686d7`) were deliberate reconnaissance runs with all 655 channels selected, made to establish which parameters the car publishes — see `logs/CHANNEL_CENSUS.md` . They are not usable as driving data and that is by design.

A4**The meeting where you explain the claim** **DONE**

Twenty minutes at a whiteboard, then questions. Five things, in order:

- The engine contains slow things that heat damages — the turbine housing, the oil.
- A controller that only sees *now* must wait for the temperature to climb before backing off. By then the damage is done.
- A controller that knows a hill is coming can protect before the heat arrives.
- How much that is worth depends on one ratio: how far ahead you see, divided by how slowly the protected thing responds. H/τ .
- Nobody has mapped how that ratio governs the payoff. That gap is the project.

FINISHED WHEN

All five can say points four and five out loud, without notes.

DO NOT

Explain Soft Actor-Critic, Lagrange multipliers or Wiebe functions in this meeting. They will not stick, and the attempt convinces four people the project is beyond them.

A5**Everyone reads two documents. Only two.** **DONE**

The novelty statement in full; the vocabulary PDF skimmed once and kept open as a dictionary.

FINISHED WHEN

Everyone can name the paper closest to this work — Liu and Seiler, on regret bounds for H^∞ preview controllers.

ONE DOCUMENT IS NOW STALE

`Supervisory_Engine_Tuner.pdf` describes the plant as a 2.0 L inline-four throughout, which it never was.
See `DOCUMENT_STATUS.md` for the full list of documents carrying void numbers.

Make the simulator match the real car

This is the phase that makes an examiner believe the rest, and it is finished. Everything after it rests on a table showing that a model written by five undergraduates behaves like an actual BMW B58.

Phase B

Six steps — all complete

B1**Look at the drive before you compute anything** DONE

Plot engine speed, manifold pressure, intake temperature, coolant, oil and vehicle speed against time on one figure.

FINISHED WHEN

You can point at the plot and say “that is where I pulled into fourth” — and be right.

RESULT, AND IT CHANGED THE WHOLE PHASE

Looking first is what caught the biggest error in the project. The channel named `Intake manifold absolute pressure` reads 13.49 psi at warm idle against an ambient of 14.23 — but a throttled engine at idle must sit near a *third* of ambient. It is a **pre-throttle sensor**. Using it as the load input gives **75 % air-mass error**; inverting the air-mass channel instead gives 1.3 %.

B2**Extract the steady operating points** DONE

`build_dataset.py` finds windows where speed and engine speed both hold still, averages each one, and de-duplicates.

FINISHED WHEN

The script prints a table of distinct operating points.

RESULT

22 distinct operating points spanning 31–82 kPa and 1549–5053 rpm, pooled across eight drives. Three filters were added, each written from a measurable defect rather than from a residual: fuel-cut windows excluded (negative spark), windows whose real wall-clock span strays from 60 s excluded, and windows containing a logger gap excluded.

B3

Score the model against those points DONE

`compare_log.py` runs the plant at each measured point and reports the disagreement.

FINISHED WHEN

Load residual is under 15 % and you can say what it does *not* cover.

RESULT

2.8 % residual over 22 points. Per drive: 3.5 % / 3.9 % / 1.8 % / 0.6 %. The residual has moved 9.8 → 4.4 → 2.3 → 2.8 % as filters were added and as an eighth drive widened the range. Report the history, not just the best number.

B4

Validate against published figures too DONE

`validate.py` checks eleven quantities against bands from the combustion literature.

FINISHED WHEN

The table regenerates and every miss has a written reason.

RESULT

8 of 11 inside band. The three misses: EGT maximum 777 °C against a 600–750 band measured at a point above this car's cruise range; and two oil rows that follow from calibrating the oil coupling against the car rather than the literature. **Do not tune to close them.** Matching the car and missing a published band is the better failure.

B5

Calibrate the baseline ECU against the real car DONE

Spark map, enrichment strategy and knock feedback, all fitted to logged data instead of guessed.

FINISHED WHEN

The baseline no longer does anything the real car demonstrably does not do.

RESULT

The guessed baseline ran a knock integral of 1.4–2.1 — detonating continuously, which no production ECU does. About 90 % of its apparent damage was that penalty. Enrichment turned out not to depend on manifold pressure *at all* (correlation –0.05 over 1055 samples); it depends on engine speed and on how long the engine has been held at load. Knock retard is now measured: p99 **9.8°** at steady gear, against the model's 12° cap.

B6

Calibrate the thermal network DONE

Drive `thermal.py` over whole logs as a time series and compare against measured coolant and oil.

FINISHED WHEN

The modelled traces track the measured ones, and every parameter is either fitted or explicitly marked as reasoned.

RESULT

Coolant RMSE 4.2 K, oil RMSE 4.1 K over 80 minutes. The oil-to-coolant coupling was wrong by an order of magnitude and is now fitted. **The radiator cannot be identified on this car at all** — every water-pump channel it publishes reads zero, so there is no coolant-flow signal, and the only observable is $\Delta T = Q/\dot{m}$ with both terms unknown. Those three parameters stay reasoned; say so in Chapter 3.

GATE B Passed

8 September 2026

YOU HAVE

A validated simulator, a calibrated baseline, a calibrated thermal network, and a written list of what none of it covers. Regenerate the evidence with `python build_dataset.py "logs/raw/*.csv"`, then `compare_log.py`, `validate.py` and `verify_docs.py`.

DO NOT REOPEN THIS PHASE

More driving data will not improve the project from here. The compressor envelope has one empty bin that no drive can fill, because the air-mass sensor saturates before reaching it. Phase B is finished; the remaining risk is all in C and D.

Get an agent to learn

The environment is built, the reward is checked and the scenario loads the engine. Nothing has been trained yet. This phase is mostly waiting for computers.

Phase C

Four steps, and the second one is the important one

C1

Confirm the environment rewards anticipation at all DONE

`check_premise.py` races a hand-written predictive policy against a reactive one, then blinds the predictive one.

FINISHED WHEN

The predictive policy beats the reactive one, and the blinded one lands on the reactive score.

RESULT

Baseline 829.2, reactive 548.6, predictive 437.6, **blinded 548.6**. Reactive cuts damage 33.9 %, predictive 47.2 % — a **13.4-point** preview advantage. Run it before writing any RL code; if this ever fails, no amount of SAC tuning will help.

C2

Check the reward cannot be gamed DONE

`test_reward.py` runs four checks: neutral scores about zero, refusing torque is punished, disabling preview changes what the agent sees, and no reward is infinite.

FINISHED WHEN

All four pass. **Do not train until they do.**

RESULT, AFTER FIXING THE HACK TWICE

Neutral -0.004 , starver **-0.280** , ablation live, all finite. The hack was real and it came back once: a policy that simply refused to make torque outscored the neutral one. The fix is a tolerance band plus a steep hinge, sized so **no achievable damage saving pays for a sustained torque shortfall beyond 10 %**. Put that sentence in Chapter 4.

RE-RUN THIS AFTER ANY CHANGE TO THE PLANT OR THE SCENARIO

Not just after reward changes. A reward is only safe relative to the dynamics it scores — that is exactly how the hack came back.

C3

Train one agent, badly, and do not care

`pip install "stable-baselines3[extra]"`, then `python train.py --steps 50000 --seed 0`. Expect a poor result. It *running* is the point.

FINISHED WHEN

A checkpoint file exists and `curve.csv` has more than one row.

BUDGET AN OVERNIGHT, NOT AN EVENING

About 4.6 hours per seed on one CPU core — measured, not estimated: the environment alone runs at 19.5 steps/s and SAC's gradient updates bring the loop to 3.0. Checkpoints land every 10 000 steps, so a closed laptop costs minutes rather than the run.

C4

Five seeds each way, one per person, overnight

`--seed 0` through `--seed 4`, then the same five again with `--no-preview`. Ten runs, five machines, two nights.

FINISHED WHEN

Ten run directories exist, each with a checkpoint and a learning curve.

ONE SEED IS NOT A RESULT

Reinforcement learning is noisy enough that a single run can show anything you like. Five seeds is the minimum that lets you report a median and an interquartile range, and Phase D needs both sets.

GATE C

Before you move on

end of week 10

YOU SHOULD HAVE

Ten trained agents — five sighted, five blinded — and a learning curve that goes up rather than sideways. The agents do not need to be good yet.

IF TRAINING WILL NOT CONVERGE

Lower the learning rate first (`--lr 1e-4`), then shorten the episode. Do not start editing the reward to make training easier — that is how the hack got in, and `test_reward.py` is the only thing standing between you and a Phase D table that means nothing.

The baselines and the ablation

This is the phase you pass on. If the semester collapses and you finish nothing else, finish this.

Phase D

Five steps to a defensible result

D1

Fix the evaluation protocol before you look at any result

Twenty episodes, chosen once: seeds, grades, ambient temperatures, preference weights. Write them into a file and commit it.

FINISHED WHEN

The twenty episodes are in version control with a commit message saying they are frozen.

THIS IS THE ONE MISTAKE THE PROJECT CANNOT RECOVER FROM

Changing the test set after seeing results invalidates everything downstream, and an examiner will ask when you chose it. Choose now, while you have no results to be tempted by.

D2

Run the three baselines on those twenty episodes

The production-representative ECU with neutral trims; the reactive protection policy; and a fixed-conservative policy that always backs off.

FINISHED WHEN

Three rows of numbers exist for the same twenty episodes.

D3

Run the five sighted agents on the same twenty

Median and interquartile range across seeds, not the best seed.

FINISHED WHEN

You can state the agent's advantage over each baseline with a spread attached.

REPORT THE MEDIAN

Quoting your best seed is the most common way an undergraduate RL result fails to replicate, and it is obvious to anyone who has trained an agent.

D4

Run the five blinded agents — the ablation

Identical in every respect except that the preview features are zeroed.

FINISHED WHEN

You have a table with sighted and blinded side by side, five seeds each.

WHAT TO EXPECT

The hand-written version of this already lands exactly: blinded predictive scores what reactive scores, to the decimal, across two engines and two scenarios. If the trained version shows the same structure, that is the project's central result. **If it does not, report that too** — a refuted ablation is still a finding, and it is more interesting than a confirmed one nobody believes.

D5

Write the table and the paragraph under it

One table, five rows. One paragraph saying what it shows and what it does not.

FINISHED WHEN

Someone outside the team reads the table and the paragraph and can say what was measured.

GATE D

The project now passes

end of week 14

YOU SHOULD HAVE

A validated simulator, an agent beating two baselines, and an ablation isolating preview. Written up, that is a complete BSc project.

EVERYTHING AFTER THIS RAISES THE CEILING; NOTHING AFTER THIS PROTECTS THE FLOOR.

A second, completely different plant

The H/ τ claim is about systems in general, not about engines. One plant cannot demonstrate a general criterion, and an examiner will say so.

Phase E

Four steps, about two hundred lines of code

E1

Write `battery.py` with the same contract

Keyword inputs in, a dictionary of named outputs out — exactly like `plant.predict()`. An equivalent-circuit cell with a thermal node: current in, terminal voltage, heat generation and cell temperature out.

FINISHED WHEN

`from battery import predict` works and returns a dictionary.

DO NOT RENAME `PREDICT`

The whole point is that the agent code drives either plant unchanged. If the contract differs, you have two projects instead of one.

E2

Give it a plausible thermal time constant, and know what it is

A cell is slower than a turbine housing and faster than an engine block. Pick from published cell data and write down the source.

FINISHED WHEN

You can state τ for the battery plant and say where the number came from.

PASS THE GEOMETRY EXPLICITLY

The engine plant taught this the hard way: a default that silently selects a different physical system is a trap. Never let `battery.py` have a “convenient” default plant.

E3

Wrap it in the same environment

Same reward structure, same preview mechanism, same ablation switch. Only the plant changes.

FINISHED WHEN

`test_reward.py` passes against the battery environment too.

E4

Train one agent on it

One seed is enough here. You are checking the pipeline transfers, not producing a result yet.

FINISHED WHEN

An agent trains on the battery plant without any change to the agent code.

The main experiment

Everything so far exists to make this figure believable. One plot: preview value against H/τ , with points from both plants on the same axes.

Phase F

Five steps to one figure

F1

Measure τ for each protected component PART DONE

Step the load, record the response, read 63.2 % off the curve.

FINISHED WHEN

Every component in the sweep has a τ with a stated method.

PARTLY DONE

Turbine housing **48.0 s** by step response, **50.3 s** by C/UA — report both and say why they differ (gas-side heat transfer rises with flow). Oil **16.0 s**. Coolant 9.5 s, short because the thermostat regulates it. The battery plant's τ is Phase E.

F2

Sweep the preview horizon at fixed τ

Vary H with the plant held constant. Several points per plant.

FINISHED WHEN

You have preview value as a function of H for at least one component.

F3

Fix the threshold rule, and state it in the methods

The constraint must bind, and bind equally, in every configuration being compared — otherwise the experiment measures the threshold rather than the policy.

FINISHED WHEN

One rule is written down and used everywhere.

THIS IS THE OPEN PROBLEM IN PHASE F

The percentile rule in `generality_test.py` assumes the temperature spends a minority of the episode near its peak. The standard scenario is a sustained climb — nine of twelve minutes at the top — so the 80th percentile lands on the peak and the top three rows saturate at 100 % for both policies. `check_premise.py` hit the same wall and solved it by anchoring to the **damage model** instead: 1123 K, the knee of the turbine damage term. That is a statement about the component, so it transfers between scenarios. **Adopt the same anchoring here.**

F4

Plot both plants on one pair of axes

Preview value against H/τ . Engine points and battery points, distinguishable but on the same figure.

FINISHED WHEN

The figure exists and you can say whether the points collapse onto one curve.

PRELIMINARY, ENGINE ONLY, FIXED-THRESHOLD SWEEP

Preview edge rises 16.5 → 18.0 → 26.0 points as H/τ falls from 4.47 to 0.60, then the component becomes so massive the constraint stops binding. Those figures were 0.0/0.1/0.2 under the old 930 K threshold. **Nothing about the plant changed — only the threshold.** Report the threshold with every preview number; an advantage quoted without the limit it was measured against is not a result.

F5

Say what the figure means, including if it disagrees with you

If the two plants do not overlap, the criterion is incomplete — and reporting that is still a contribution.

FINISHED WHEN

One paragraph states the finding and its limits, and survives being read aloud to someone hostile.

Write it and defend it

The writing is easier than you expect, because the numbers already exist and every one of them is printed by a script.

Phase G

Five steps, and one table to memorise

G1**Run `verify_docs.py` before writing a single number**

It recomputes twenty-two published figures from the shipped data and fails if any has drifted.

FINISHED WHEN

It prints “All 22 figures match the documents”.

IT EXISTS BECAUSE FIVE FIGURES HAD ALREADY DRIFTED

Four for the same reason: a number computed on one drive and quoted as if it were pooled. None changed a conclusion; all would have been defended in a viva.

G2**Write Chapter 3 from `validation_table.md`**

It already contains the tables, the exclusion rules and the stated limits, in the order a methods chapter wants them.

FINISHED WHEN

Every number in Chapter 3 traces to a script that still runs.

G3**Write the limitations section honestly, and early**

Peak power is not a prediction. Validation covers 31–82 kPa. The air-mass sensor saturates at 1020 kg/h, so the compressor envelope above 0.303 kg/s is unmeasured rather than sparse. The radiator cannot be identified on this vehicle. Oil above 107 °C is extrapolation. Enrichment uses dwell as a proxy for a turbine temperature the car does not publish.

FINISHED WHEN

Someone reading only the limitations section still believes the results.

WRITE THIS BEFORE THE RESULTS CHAPTER, NOT AFTER

A limitations section written last reads like an apology. Written first, it reads like control of the material — and it is the single strongest signal available to an undergraduate project.

G4

Memorise one table and one sentence

The table is Phase D's. The sentence is: *"disabling preview collapses the predictive policy onto the reactive one exactly, so whatever gap exists is attributable to preview information and nothing else."*

FINISHED WHEN

All five can say the sentence and read the table without notes.

G5

Rehearse the hostile questions

Four of them, and you already have the answers. *How do you know the simulator is right? Why should I believe one seed? Where did the test episodes come from? What would have falsified your claim?*

FINISHED WHEN

Each of the five can answer all four without looking anything up.

5 · Rhythm

A normal week, three hours

One ninety-minute session together, and roughly ninety minutes each alone. That is enough, provided the together-session always ends with each person knowing the one thing they own next.

- **First twenty minutes.** Each person says what they ran and what it printed. Not what they intend to do.
- **Middle fifty.** The current step, worked on together at one screen. Whoever owns it drives.
- **Last twenty.** Assign the next step to a named person with a finish test they can check alone.

Change one thing, re-run, write down what happened. Two changes at once and you no longer know which one did it. Every one of the six corrections in this project was found because something was changed alone and re-measured.

6 · Triage

When something breaks

SLOW	Training is far slower than the estimate	expected: 4.6 h
DRIFT	A number in a document disagrees with the code	verify_docs.py
HACK	The agent scores well but does nothing sensible	test_reward.py
FLAT	The learning curve goes sideways for 20k steps	lower --lr
DATA	A new drive adds zero operating points	see below

A drive that adds no points is a driving problem, not a code one — it was not steady enough, or the logger dropped samples. `build_dataset.py` prints which windows it rejected and why. A drive logged with hundreds of channels selected will lose a quarter of its samples to logger stalls; that is fine for a channel census and useless for steady-state data.

When to stop and ask. If two hours of work has not moved a number, stop and write down what you tried. Bring that to the next session. The written attempt is worth more than the extra two hours.

7 · Definition of done

What each piece looks like finished

A	Five identical environments, real driving data, the claim said aloud	done
B	2.8 % load residual, 8 of 11 published quantities, thermal network calibrated	done
C	Ten trained agents, five sighted and five blinded, curves that rise	next
D	One table: three baselines, sighted and blinded, median and IQR over five seeds	the floor
E	<code>battery.py</code> driving the unchanged agent code	—
F	One figure, both plants, threshold rule stated in the methods	—
G	A thesis whose every number a script still prints	—

The project’s real advantage is that almost every number came out of running something. Two calibrations were corrected against measurement, one hypothesis was refuted and reported, one whole phase was recomputed after the simulated engine turned out to be the wrong size, and the validation table states what it does not cover. Keep that. An examiner trusts a student who reports against themselves, and punishes a table that implies coverage it does not have.